

python

1. what is python and what are its key features.

Python is a high-level, interpreted, object-oriented, dynamically typed programming language.

Key features:-

1. Easy to read and write
 2. Cross-platform
 3. Interpreted
 4. Dynamically typed.
 5. Support multiple paradigms (oop, functional, procedural).
 6. Huge standard library & ecosystem.
 7. Open source.
2. what is the dif btw python 2 and python 3.?
1. python 2 is legacy; python 3 is the present and future.

python 2	python 3
1. python 2 is legacy. 2. print $\rightarrow$ statement in python 2 3. <u>Integer division</u> :- $\frac{5}{2} \rightarrow 2$ in python 2. 4. Unicode: default str in separate in python 2.	1. python 3 is the present and future. 2. function in python 3 3. exact ans 2.5 in python 3 4. default str in python 3.

python 2 stopped support in 2020.

3. Is python interpreted (or) compiled?

Python is interpreted (runs line by line using python).

→ code → Bytecode (.pyc) → executed by the PVM.

→ Not compiled into machine code like C++.

4. what are the different data types in python.

1. Numeric: int, float, complex.

2. Sequence: list, tuple, range.

3. Text: str.

4. Set types: set, frozenset.

5. Mapping: dict

6. Boolean: bool.

7. None type: None type.

5. what is the diff b/w is and ==?

1. is compares identity (memory address).

2. == compares value (contents).

Syntax:-

a = [1, 2] b = [1, 2]

a == b # True

a is b # False.

6. what are variables in python?

Variables are names that reference objects in memory.

x = 10.

7. what is the purpose of None?

Represents absence of value or null.

8. what is dynamic typing?

1. variable types are decided at runtime.

2. no need to declare types explicitly.

x = 5 # int

x = "hey" # str

9. what is the diff btw mutable and immutable types.

mutable : can be changed after creation
(list, dict, set).

Immutable: cannot be changed after creation
(str, tuple, int).

10. what are python's immutable types.

int, float, complex, bool, str, tuple, frozenset, bytes.

11. what is a keyword in python?

Reserved word with special meaning (e.g.,

if, else, def, class, is, in, for

12. what is PEP 8?

Python Enhancement proposal 8 -> official style guide for writing clean python code.

13. what is the use of id() function?

Returns the memory address (identity) of an object.

14. How does python manage memory?

1. Uses private heap space for objects.
 2. managed by python memory manager.
 3. Garbage collection via reference counting + cyclic gc.
15. what are the rules for naming variables.

1. start with letter or _.
2. followed by letters, digits, or _.
3. case sensitive.
4. cannot be a keyword.

16. what is the diff b/w a script and a module.

Script: - file intended to be executed

module: file intended to be imported and reused.

17. what are comments and docstrings in python?

Comments: # single-line, """ """ multi-line (ignored by interpreter).

Docstrings: special strings inside functions/classes

modules for documentation (help() displays them.)

18. what are literals in python?

fixed values in source code - numbers, strings, lists, dicts, etc.

Example: 10, "hello", [1, 2, 3].

19. what are magic constants like `--name--` used for?

1. `--name--` → holds module name.
 2. `--main--` if run directly.
 3. module name if imported.
20. what is the purpose of indentation in python?

Indentation defines block structure.

21. what is a namespace?

A mapping of names to objects.

Types: Local, enclosing, Global, Built-in

22. what is the `type()` function?

Returns the type/class of an object.

23. what is a dynamic language?

language where types are checked at runtime, not compile time.

24. How does python handle variable swapping?
python allows tuple unpacking.

`a, b = b, a`

25. what does `pass` do in python?

`pass` is a placeholder statement that does nothing.

It is used to define empty loops, functions, or classes without causing errors.

26. what are the internal steps in python code execution?

1. lexing $\rightarrow$ tokens
2. parsing $\rightarrow$ Abstract Syntax tree (AST).
3. compilation $\rightarrow$ Bytecode (.pyc)
4. Execution $\rightarrow$ python virtual machine runs it.

27. How does python generate and use .pyc files?

1. python compiles .py to .pyc (bytecode)
2. Stored in --pycache--
3. speeds up subsequent imports.

28. what is the role of python Virtual machine (PVM)?

The PVM interprets and executes python bytecode line by line. It acts as the run-time engine that makes python code platform independent.

29. what is the Abstract Syntax Tree (AST)?

AST is a tree representation of source code structure. It helps the python interpreter analyze program logic before converting it into bytecode.

30. what is the internal memory layout of python objects.

Every python object has three main parts.

- * Type pointer (points to object class).
- * Reference Count (for memory management).

* Actual data (objects value).

31. what is the Global Interpreter Lock (GIL) in python?

The GIL is mutex that allows only one thread to execute python byteCode at a time.

32. How does python perform garbage collection?

Python automatically frees memory of objects no longer in use.

* Reference counting (immediate cleanup).

* cyclic garbage collector for circular references to handle.

33. what are reference counting and cyclic gc in python?

→ Reference counting - deletes object when no references exist.

→ cyclic gc - detects & clears cycles (obj1 → obj2 → obj1)

34. where are functions and variables stored during execution (stack vs heap)?

Variables & objects → heap.

function calls / frames → stack.

35. How are primitive data types handled in memory?

1. Immutable primitives (int, float, bool, str) are stored as objects in heap.

2. Small integers & short strings are interned (cached for reuse).

Control flow :-

36. What are the control flow statements in python?

Control flow statements decide the order in which code executes. Examples include if-else, for and while loops, which programs make decisions and repeat actions.

37. What is the diff b/w if, elif and else?

if :- Executes a block when a condition is true.

elif :- Checks another condition if the previous if or elif was false.

else :- Executes when all previous conditions are false.

38. What is the diff b/w while and for loops?

for loop: Iterates over a sequence (like list, string, range).

while loop: - Repeats as long as a condition is true.

39. What is the use of break and continue?

Break: Exits the loop immediately.

Continue: Skips the current iteration and moves to the next.

40. What is the use of else block with loops?

The else block runs when the loop completes normally (without a break). It is used to check if the loop runs to the end.

41. How can we loop over a range of numbers?

Use range() function with a for loop.

for i in range(5): iterates over: 0, 1, 2, 3, 4.

42. What is the use of range()?

range() generates a sequence of numbers.

It is commonly used for looping a fixed no. of items.

43. What is the diff b/w range() and enumerate()?

* range(): Generates no. for iteration.

* enumerate(): Returns index and value together when iterating over a sequence.

44. What is zip() used for in loops?

zip() combines multiple sequences element wise, letting you loop over them in parallel.

45. What

A nested

loops

outer

46. What

Loop

condition

if not

47. How

pyth

Use

mea

48. The

val

Val

even

out

49. W

Op

A

It

Com

50. H

Pe

so

45. what is nested loop?

A nested loop is a loop inside another loop. Inner loops run completely for every iteration of the outer loop.

46. what are infinity loops?

Loops that never stop because the exit condition is never met. They can crash programs if not handled properly.

47. How to find if a number is even or odd in python?

Use the modulus operator: if `num % 2 == 0`: means the number is even, else it's odd.

48. How does python evaluate truthly and falsy values?

Values like 0, None, '', [], {} are false; everything else is truthly. They decide condition outcomes in if statements.

49. what is a conditional expression (ternary operator)?

A one-line if-else: `x = a if condition else b`.

It assigns a value based on a condition in a compact form.

50. How do you use short circuiting in python?

Python stops evaluating and/or expressions as soon as the result is determined. for example,

in x and y, if x is false, y is not checked.

51. what are operators in python?

Python has arithmetic, comparison, logical, assignment, bitwise, identity and membership operators. Each type performs specific operations on values or values.

52. what is the diff b/w ^{identity} and vs equality in python?

- * Identity (is): checks if two objects occupy the same memory.
- * Equality (==): checks if two objects have the same value.

53. How does python handle operator precedence?

Operator precedence determines the order in which operations are performed. parenthesis can override the default precedence.

54. what are logical operators in python?

logical operators are and, or, not. They combine or invert Boolean values in conditional statements.

55. what is membership testing (in, not in)?

In:- checks if a value exists in a sequence, like a list or string.

not in:- checks that value does not exist.

56. what is the diff btw += and =+?

+= adds the right hand value to the left hand variable and updates it.

=+ is just = with unary + and does not increment.

57. how does the not operator work?

not inverts the Boolean value: True becomes false and vice versa. Its used to reverse conditions.

58. what is the diff btw bitwise and logical operators.

Bitwise operators:- (&, |, ^, ~) works on bits of integers.

logical operators:- (and, or, not) work on boolean values.

59. can you overload operators in python?

Yes python allows operator overloading by defining special methods like --add-- i.e. --eg-- __add__ -- mul -- etc., in a class.

functions

60. what is a function in python?

A function is a reusable block of code that performs a specific task. It helps organize programs and avoid repetitions.

61. How do you define a function?

Use def keyword with name, parameters, body

def func_name (params):

62. what is the use of return?
return sends a value back to the caller without it a function returns non by default.

63. what are args and * keyword args?
*args: allow passing any no. of positional arguments.

**kwargs: - allows passing any no. of keyword arguments

64. what is a default argument?

A default argument provides a value if the caller does not supply one:

```
def greet (name="Guest");  
    print (name)
```

65. what is a keyword-only argument?

Arguments that must be passed using their parameter name after * in function definition.

66. what is lambda function?

A small anonymous function defined with lambda.

It can take any no. of arguments but only one expression.

67. what is the diff btw yield and return?

return: Send a value and exists the function.

yield: produces a value and pauses the function creating a generator.

68. what variable only a (LEGB)

69. what global

non local

70. how simply def

def

fun (

71. what small is

small

is

72. A fe

A fe

-ts

73. c

* n

* fi

68. what is the scope of variables in functions?
variables inside a function are local. They exist only outside the function declare global, Built-in (LEGB rule).

69. what are global and non local keywords?

global:- access global variables, function modify global variable

non local:- let a nested function modify a variable from the enclosing scope.

70. how do you pass a list to a function?
simply pass it as an argument.

```
def func (list):  
    print(list)
```

```
func([1,2,3])
```

71. what is recursion?

Recursion is a functions calling itself to solve smaller instance of a problem until a base case is reached.

72. what is a higher-order function?

A function that takes other functions as argument or returns a function.

73. what are map(), filter(), and reduce()?

* map(): applies a function to every item in a sequence.

* filter(): selects items based on a condition.

* Reduce(): (from functools) reduces a sequence to a single value.

74. what is the use of enumerate() in function, enumerate() returns both index and value while iterating over a sequence.

75. what is a closure in python?

A closure is a nested function that remembers variables from its enclosing scope, even after the outer function ends.

76. what is a decorator?

A decorator is a function that modifies another function without changing its code. It's used with @decorator_name.

77. Can a function return another function?

Yes, functions in python are objects, so one function can return another function for dynamic behaviour.

78. How to define an anonymous function? use lambda keyword;

```
square = lambda x: x**2
```

```
print(square)
```

79. what is function caching in python?

caching (like functools.lru_cache) store results of expensive function calls to reuse them later and

improve

Data st

80. wha

hist :m

tuple:

81. wha

lists

-nts o

82. wh

A cor

[x*

83. w

A

un

84. w

85. w

A

ele

86. th

sequence
function,
while
members
after
as
dynamic
of
and

improve performance.

Data Structures

80. what is diff btw list and tuple?

list: mutable, can be changed after creation

tuple: Immutable, cannot be changed once created and is faster.

81. what are the key features of lists?

lists are ordered, mutable, allow duplicate elements and can store mixed data types.

82. what are list comprehensions?

A concise way to create lists using a single line
[x**2 for x in range(5)]

83. what is a dictionary?

A dictionary stores key-value pairs; keys are unique and values can be any type.

84. what is diff btw list and set?

list: ordered, allows duplicates

set: unordered, does not allow duplicates.

85. what is a set? what are its use cases?

A set is a unordered collection of unique elements useful for removing duplicates or performing union/intersection operations.

86. How do you remove duplicates from a list?

convert the list to a set and back:

List(set(my_list)).

del(): Deletes element or variable entirely.

94. what is the use of slicing?

slicing extracts a portion of a sequence:

my-list[1:5] gets elements from index 1 to 4.

95. what is object-oriented programming?

OOP is a programming paradigm that organizes code into objects, combining data and behaviour.

It supports encapsulation, inheritance, and polymorphism.

96. what are classes and objects in python?

class: A blue print for creating objects.

object: An instance of a class with its own data and behaviour.

97. what is `--init--()`?

`--init--` is a constructor method that initializes an object's attributes when it is created.

98. what is `self` in a class?

`self` represents the current instance of the class. It is used to access instance variables and methods.

99. what is the difference between instance variables and class variables?

Instance variables: - Belong to an object and are unique for each instance.

Class variables: shared across all instances of the class.

100. what is inheritance?

Inheritance allows a child class to reuse attributes and methods of another class parent, promoting code reuse.

101. what is multiple inheritance?

A class inherits from more than one parent class, combining behaviors from all parents.

102. what is method overriding?

A child class redefines a method from its parent class to change or extend behavior.

103. what is `super()` used for?

`super()` allows calling parent class methods from a child class commonly used in constructors.

104. what are mg magic methods?

special methods with double underscores like `--init--`, `--str--`, or `--add--` that define object behavior in python.

105. what is `--str--` & `--repr--`?

⇒ `--str--`: Returns a user-friendly string of an object.

⇒ `--repr--`: Returns a developer friendly string, ideally one that can recreate the object.

106. what is encapsulation objects

exposing 107. what polymorphism

to the len()

108. what static class

109. what pr

⇒ pu

⇒ pe

⇒ p

106. what is encapsulation?

Encapsulation restrict direct access to an objects data using private / protected variables, exposing only necessary methods.

107. what is polymorphism in python?

polymorphism allows different objects. to respond to the same method call in different ways e.g. `len()` works on strings, lists and tuple.

108. what are static methods and class methods?

static method: Defined with `@static method` does not access class / instance data.

class method: Defined with `@ class method`, takes the class as the first argument (`cls`)

109. what are access modifiers (public, private, protected) in python?

⇒ public: Accessible anywhere.

⇒ protected (`--var`): should not be accessed the class but still possible.

⇒ private (`--var`): Name-mangled to restrict access from outside the class.